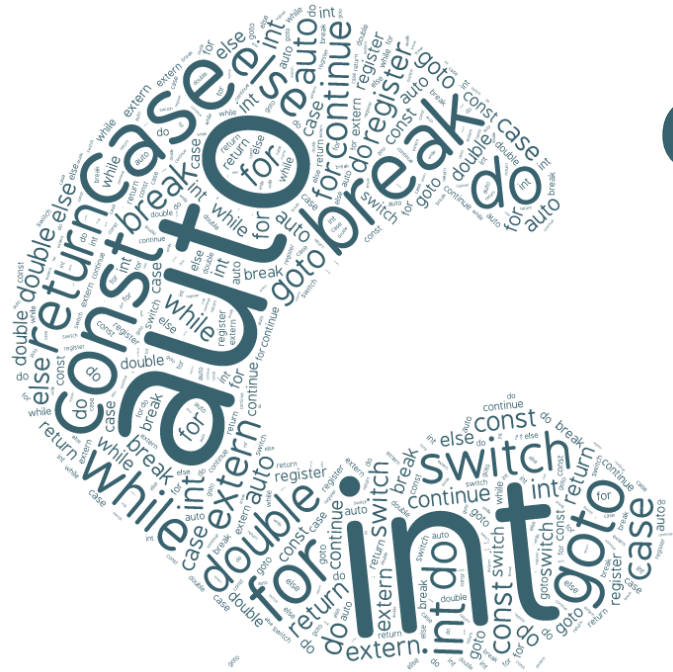




chapter 5

■ ■ ■ ■ ■

数组和广义表

- 5.1 数组的定义
- 5.2 数组的顺序存储表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的定义
- 5.5 广义表的存储结构

数组： 由一组名字相同、下标不同的变量构成

数组的特点：

- ① 数组中各元素具有**统一的类型**；
- ② 数组元素的下标一般具有**固定的上界和下界**，即数组一旦被定义，它的维数和维界就不再改变。
- ③ 数组的**基本操作比较简单**，除了结构的初始化和销毁之外，只有存取元素和修改元素值的操作。

一维数组的特点：1个下标， a_i 是 a_{i+1} 的直接前驱

二维数组的特点：2个下标，每个元素 $a_{i,j}$ 受到两个关系（行关系和列关系）的约束：

$$A_{mn} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

一个 $m \times n$ 的二维数组可以看成是 m 行的一维数组，或者 n 列的一维数组。

N 维数组的特点： n 个下标，每个元素受到 n 个关系约束

一个 n 维数组可以看成是由若干个 $n-1$ 维数组组成的线性表。

N 维数组的数据类型定义

$$n_ARRAY = (D, R) \quad \text{其中:}$$

数据对象: $D = \{a_{j_1, j_2, \dots, j_n} \mid j_i \text{ 为数组元素的第 } i \text{ 维下标, } a_{j_1, j_2, \dots, j_n} \in \text{Elemset}\}$

数据关系: $R = \{R_1, R_2, \dots, R_n\}$

$$R_i = \{ \langle a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_{i+1}, \dots, j_n} \rangle \mid a_{j_1, j_2, \dots, j_i, \dots, j_n}, a_{j_1, j_2, \dots, j_{i+1}, \dots, j_n} \in D \}$$

基本操作: 构造数组、销毁数组、读数组元素、写数组元素

数组的抽象数据类型定义略

- 5.1 数组的定义
- 5.2 数组的顺序存储表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的定义
- 5.5 广义表的存储结构

问题：计算机的存储结构是一维的，而数组一般是多维的，怎样存放？

解决办法：事先约定**按某种次序**将数组元素排成一列序列，然后将这个线性序列存入存储器中。

在二维数组中，既可以规定按行存储，也可以规定按列存储。

注意：

- 若规定好了次序，则数组中任意一个元素的存放地址便有规律可寻，可形成地址计算公式；
- 约定的次序不同，则计算元素地址的公式也有所不同；
- C和PASCAL中一般采用**行优先**顺序；FORTRAN采用**列优先**。

无论规定行优先或列优先，只要知道以下三要素便可随时求出任一元素的地址（意义：数组中的任一元素可随机存取）：

- ① 开始结点的存放地址（即基地址）
- ② 维数和每维的上、下界；
- ③ 每个数组元素所占用的单元数

$$A_{mn} = \begin{bmatrix} a_{c1,c2} & \dots & a_{c1,d2} \\ \dots & a_{ij} & \dots \\ a_{d1,c2} & \dots & a_{d1,d2} \end{bmatrix}$$

补充：计算二维数组元素地址的通式

设一般的二维数组是 $A[c_1..d_1, c_2..d_2]$ ，这里 c_1, c_2 不一定是0或1

则行优先存储时的地址公式为：

$$LOC(a_{ij}) = LOC(a_{c_1, c_2}) + [(i - c_1) \times (d_2 - c_2 + 1) + (j - c_2)] \times L$$

数组基址

a_{ij} 之前的行数

总列数，即
第2维长度

a_{ij} 本行前面
的元素个数

单个元素
长度

二维数组列优先存储的通式为：

$$LOC(a_{ij}) = LOC(a_{c_1, c_2}) + [(j - c_2) \times (d_1 - c_1 + 1) + (i - c_1)] \times L$$

【例】如何求出 $a(3,2)$ 的存储地址？

	0	1	2	3
0	$a(0,0)$	$a(0,1)$		$a(0,3)$
1	$a(1,0)$	$a(1,1)$		$a(1,3)$
2				
3			$a(3,2)$	
4				
5				
6	$a(6,0)$			$a(6,3)$

要事先确定：

- ①是行优先方式还是列优先方式？
- ②数组的首地址是多少？
- ③每个元素的长度？

【例】(软考题): 一个二维数组A, 行下标的范围是1到6, 列下标的范围是0到7, 每个数组元素用相邻的6个字节存储, 存储器按字节编址。那么, 这个数组的体积是_____个字节。

答: $\text{Volume} = m \times n \times L = (6-1+1) \times (7-0+1) \times 6 = 48 \times 6 = 288$

【例】已知二维数组 $A_{m,m}$ 按行存储的元素地址公式是:
 $\text{LOC}(a_{ij}) = \text{LOC}(a_{11}) + [(i-1) \times m + (j-1)] \times K$, 请问按列存储的公式相同吗?

答: 尽管是方阵, 但公式仍不同。应为:

$$\text{LOC}(a_{ij}) = \text{LOC}(a_{11}) + [(j-1) \times m + (i-1)] \times K$$

若是 N 维数组，其中任一元素的地址该如何计算？

教材已给出低维优先的地址计算公式
该式称为 n 维数组的映像函数：

$$\text{LOC}(j_1, j_2, \dots, j_n) = \text{LOC}(0, 0, \dots, 0) + \sum_{i=1}^n C_i j_i$$

数组基址

前面若干元素占用的地址字节总数

其中 $C_n = L$, $C_{i-1} = b_i \times C_i$, $1 < i \leq n$

每个元素长度

第 i 维长度

与所存元素个数有关的系数，可用递推法求出

三维数组且列优先时的元素地址要会计算！

N维数组的顺序存储表示

```
#define MAX_ARRAY_DIM 8 //假设最大维数为8

typedef struct{
    ELemType *base;      //数组元素基址
    int      dim;        //数组维数
    int      *bound;     //数组各维长度信息保存区基址
    int      *constants; //数组映像函数常量的基址
}Array;
```



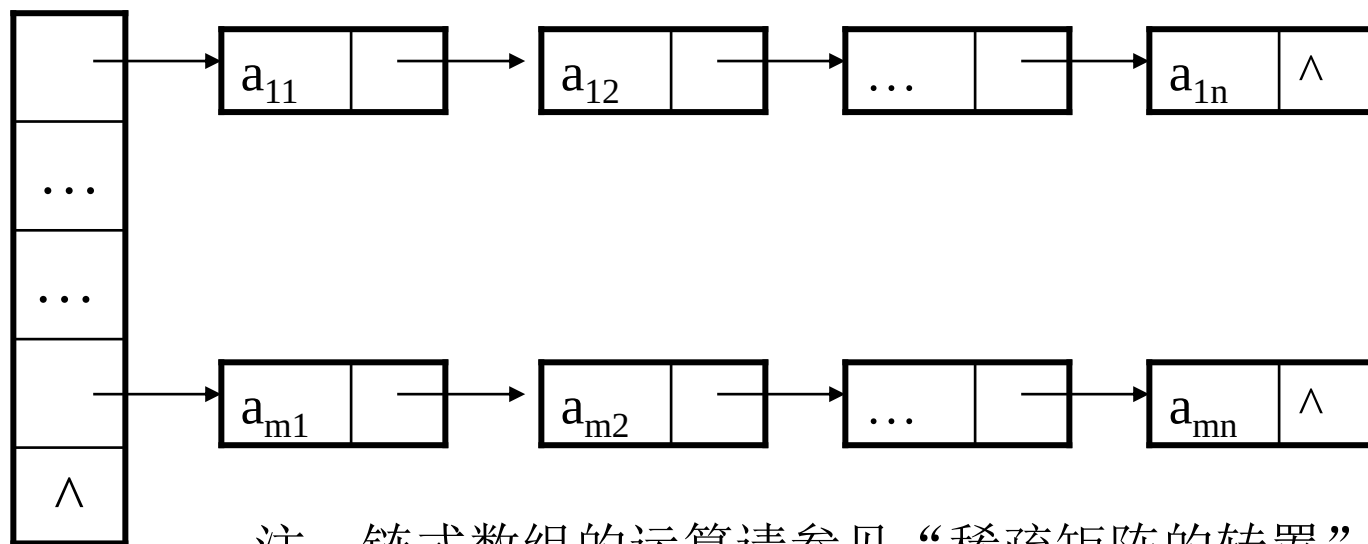
即 C_i 信息保存区

数组的基本操作函数说明（有5个）

注意：本章所讨论的数组与高级语言中的数组有所区别：高级语言中的数组只是顺序结构；而本章的数组既可以是顺序的，也可以是**链式结构**，用户可根据需要选择。

补充：**数组的链式存储方式**—用**带行指针向量的单链表**来表示。

行指针向量



注：链式数组的运算请参见“稀疏矩阵的转置”

- 5.1 数组的定义
- 5.2 数组的顺序存储表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的定义
- 5.5 广义表的存储结构

讨论：

1. 什么是压缩存储？

若多个数据元素的值都相同，则只分配一个元素值的存储空间，且零元素不占存储空间。

2. 所有二维数组（矩阵）都能压缩吗？

未必，要看矩阵是否具备以上压缩条件。

3. 什么样的矩阵具备以上压缩条件？

一些特殊矩阵，如对称矩阵，对角矩阵，三角矩阵，稀疏矩阵等。

4. 什么叫稀疏矩阵？

矩阵中非零元素的个数较少（一般小于5%）

重点介绍稀疏矩阵的压缩和相应的操作。

一、稀疏矩阵的压缩存储

问题：

如果只存储稀疏矩阵中的非零元素，那这些元素的**位置信息**该如何表示？

解决思路：

对每个非零元素**增开**若干存储单元，用来存放其所在的**行号**和**列号**，便可准确反映该元素所在位置。

实现方法：

将每个非零元素用一个三元组 (i, j, a_{ij}) 来表示，则每个稀疏矩阵可用一个**三元组表**来表示。

三元素组表中的每个结点 (i, j, a_{ij}) 对应于稀疏矩阵的一个非零元素，它包含有三个数据项，分别表示该元素的行下标、列下标和元素值。

【例】写出右图所示稀疏矩阵的压缩存储形式。

至少有4种存储形式：

方法1：用线性表表示

方法2：用十字链表表示

方法3：用三元组矩阵表示

方法4：用带辅助向量的三元组表示

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

方法1：用线性表表示

$((1,2,12), (1,3,9), (3,1,-3), (3,5,14), (4,3,24), (5,2,18), (6,1,15), (6,4,-7))$

```
typedef struct{
    Triple data[MAXSIZE+1]; // 三元组表，以行为主序存入一维向量data[ ]中
    int mu;           // 矩阵总行数
    int nu;           // 矩阵总列数
    int tu;           // 矩阵中非零元素总个数
}TsMatrix;
```

对三元组表data[]的整体定义

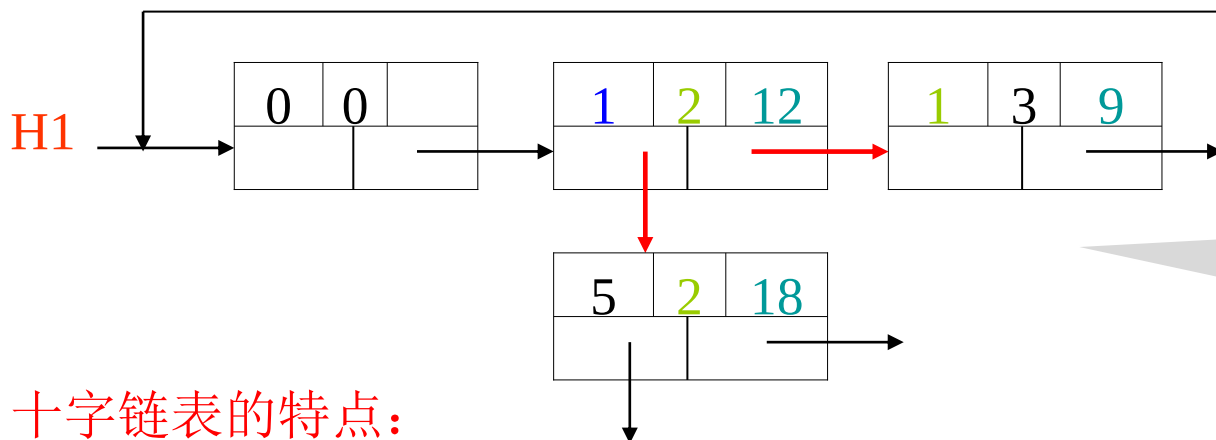
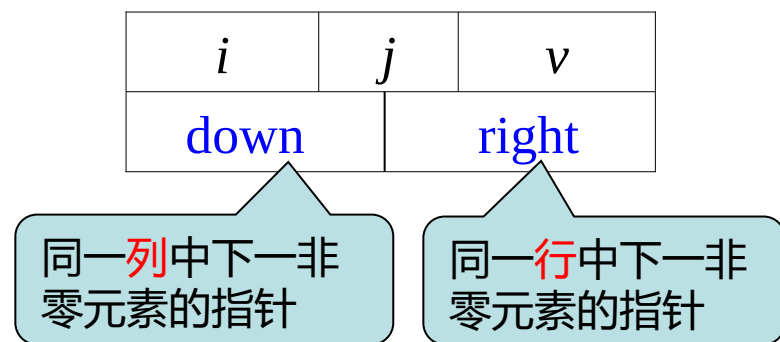
```
#define MAXSIZE 125000 // 设非零元素最大个数125000
typedef struct{
    int i;           // 元素行号
    int j;           // 元素列号
    ElemType e;      // 元素值
}Triple;
```

对表中每个结点的结构定义

方法2：用十字链表表示

用途：方便稀疏矩阵的加减运算

方法：每个非0元素占用5个域



稀疏矩阵的加减运算容易实现

十字链表的特点：

- ① 每行非零元素链接成带表头结点的循环链表；
- ② 每列非零元素也链接成带表头结点的循环链表。

则每个非零元素既是行循环链表中的一个结点；又是列循环链表中的一个结点，即呈十字链状。

方法3：用三元组矩阵表示

	i	j	value
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

$$\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix}$$

注意：为更可靠描述，通常再加一行“总体”信息：即总行数、总列数、非零元素总个数

稀疏矩阵压缩存储的缺点：

将失去随机存取功能！

5.3

矩阵的压缩存储

【例】下面的三元组表表示一个稀疏矩阵，试还原出它的稀疏矩阵。

	i	j	value
0	6	4	6
1	1	2	2
2	2	1	12
3	3	1	3
4	4	4	4
5	5	3	6
6	6	1	16

$$\begin{pmatrix} 0 & 2 & 0 & 0 \\ 12 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 4 \\ 0 & 0 & 6 & 0 \\ 16 & 0 & 0 & 0 \end{pmatrix}$$

方法4：用带辅助向量的三元组表示

用途：便于高效访问稀疏矩阵中任一非零元素。

方法：增加2个辅助向量：

- ① 记录每行非0元素个数，用NUM(i)表示；
- ② 记录稀疏矩阵中每行第一个非0元素在三元组中的行号，用POS(i)表示。

i	1	2	3	4	5	6
NUM(i)	2	0	2	1	1	2
POS(i)	1	3	3	5	6	7

POS(i)如何计算？

POS(1)=1, POS(i)=POS(i-1)+NUM(i-1)

0	12	9	0	0	0
0	0	0	0	0	0
-3	0	0	0	14	0
0	0	24	0	0	0
0	18	0	0	0	0
15	0	0	-7	0	0

	i	j	v
0	6	6	8
1	1	2	12
2	1	3	9
3	3	1	-3
4	3	5	14
5	4	3	24
6	5	2	18
7	6	1	15
8	6	4	-7

二、稀疏矩阵的操作

(以转置运算为例，加减用十字链表)

目的：

$$\mathbf{M} \begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix} \xrightarrow{\text{转置后}} \begin{pmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \mathbf{T}$$

已知
三元组表
a.data

(1, 2, 12)
(1, 3, 9)
(3, 1, -3)
(3, 5, 14)
(4, 3, 24)
(5, 2, 18)
(6, 1, 15)
(6, 4, -7)

求三元组表
b.data

(1, 3, -3)
(1, 6, 15)
(2, 1, 12)
(2, 5, 18)
(3, 1, 9)
(3, 4, 24)
(4, 6, -7)
(5, 3, 14)

提问：若采用三元组压缩技术存储稀疏矩阵，只要把每个元素的行下标和列下标互换，就完成了对该矩阵的转置运算，这种说法正确吗？

答：肯定不正确！

除了：（1）每个元素的行下标和列下标互换（即三元组中的*i*和*j*互换）；

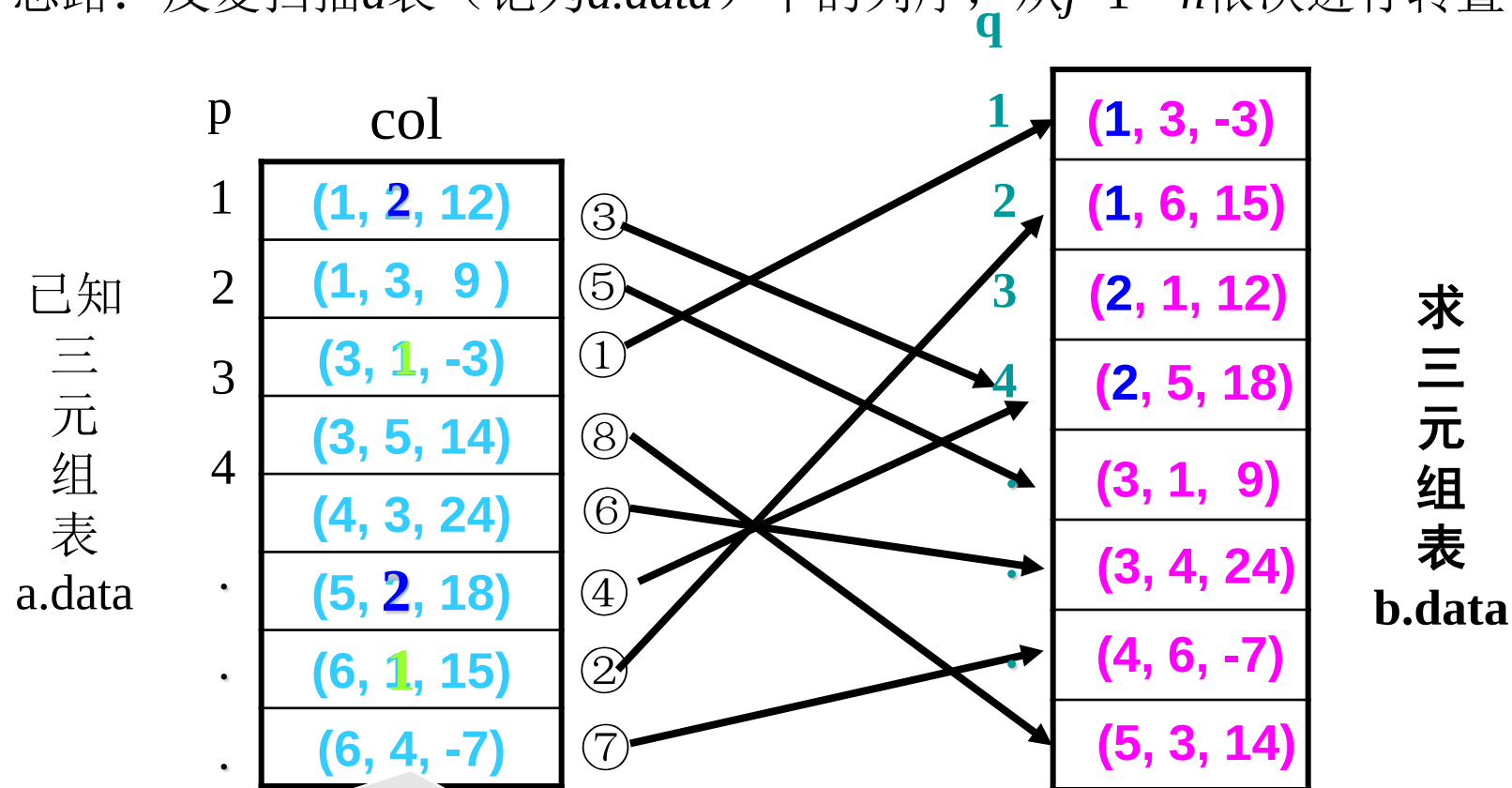
还需要：（2）T的总行数*mu*和总列数*nu*也要互换；
（3）**重排三元组内各元素顺序**，使转置后的三元组也按行（或列）为主序有规律的排列。

上述（1）和（2）容易实现，难点在（3）。

有两种实现
转置的方法 { 压缩转置
快速(压缩)转置

方法1：压缩转置

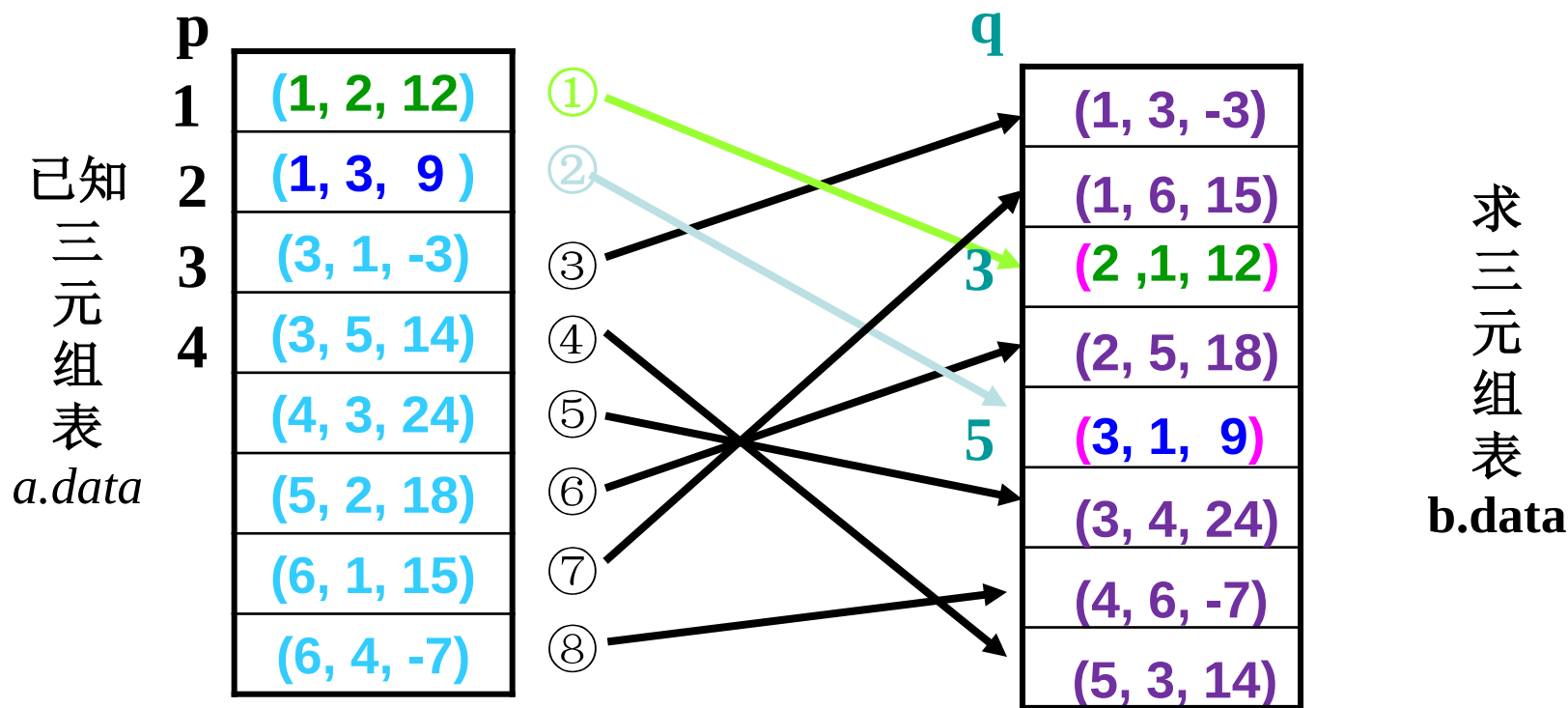
思路：反复扫描 a 表（记为 $a.data$ ）中的列序，从 $j=1\sim n$ 依次进行转置。



每个元素的列分量表示为： $a.data[p].j$

方法2：快速转置

思路：依次把 $a.data$ 中的元素直接送入 $b.data$ 的恰当位置上（即 M 三元组的 p 指针不回溯）。



关键：怎样寻找 $b.data$ 的“恰当”位置？

设计思路:

如果能**预知**M矩阵**每一列**(即T的每一行)的**非零元素个数**,
又能很快得知**第一个非零元素**在b.data中的**位置**,
则扫描a.data时便可以将每个元素准确定位(**因已知若干参考点**)

请注意a.data特征: 每列首个非零元素必定先被扫描到。

技巧: 为实现转置运算, 应当**按列**生成M矩阵三元组表的**两个辅助向量**,
让它携带每列的非零元素个数 **NUM(i)** 以及每列的第一个非零元素在
三元组表中的位置**POS(i)**等信息。

辅助向量的样式:

计算式: $POS(1)=1$

$POS(i)=POS(i-1)+NUM(i-1)$

i	1	2	3	4	5	6
NUM(i)	2	2	2	1	1	0
POS(i)	1	3	5	7	8	9

M矩阵中的列变量用col表示;

num[col]: 存放M中第col列中非0元素个数

cpos[col]: 存放M中第col列的第一个非0元素的位置
(即b.data中待计算的“恰当”位置所需参考点)

col	1	2	3	4	5	6
num[col]	2	2	2	1	1	0
cpos[col]	1	3	5	7	8	9

计算式: $\text{cpos}(1)=1$

$\text{cpos}[\text{col}] = \text{cpos}[\text{col}-1] + \text{num}[\text{col}-1]$

讨论: 求出按列优先的辅助向量后, 如何实现快速转置?

由a.data中每个元素的列信息, 可以直接从辅助向量cpos[col]中查出在b.data中的“基准”位置, 进而得到当前元素之位置。

p/t	col
1	(1, 2, 12)
2	(1, 3, 9)
3	(3, 1, -3)
4	(3, 5, 14)
	(4, 3, 24)
	(5, 2, 18)
	(6, 1, 15)
	(6, 4, -7)

三元组表

a.data .

Status **FastTransposeSMatrix** (TSMatirx M, TSMatirx &T)

{ //M是顺序存储的三元组表, 求M的转置矩阵T

T.mu = M.nu ; T.nu = M.mu ; T.tu = M.tu ;

if (T.tu) {

for(col = 1; col <=M.nu; col++) num[col] = 0; //先清0, 再统计每列非零元素个数

for(t = 1; t <=M.tu; t++) {col = M.data[t].j ; ++num [col] ;}

cpos[1] = 1; //再生成每列首元位置辅助向量

for(col = 2; col <=M.nu; col++) cpos[col] = cpos[col-1] + num [col-1] ;

for(p = 1; p <=M.tu ; p ++) //p指向a.data, 循环次数为非0元素总个数tu

{ col = M.data[p].j ; q = cpos [col] ; //查辅助向量得q, 即T中位置

T.data[q].i = M.data[p].j;

T.data[q].j = M.data[p].i;

T.data[q].value = M.data[p].value;

++ cpos[col] ;

} //for

} //if

return OK;

} //FastTranposeSMatrix;

前3个for循环用来产生两个辅助向量

元素转置

快速转置算法描述

快速转置算法的效率分析：

1. 与常规算法相比，附加了生成辅助向量表的工作。
增开了2个长度为列长的数组(`num[ ]`和`cpos[ ]`)。
2. 从时间上，此算法用了4个并列的单循环，而且其中前3个单循环都是用来产生辅助向量表的。

```
for(col = 1; col <= M.nu; col++){ }; 循环次数 = nu (列数) ;  
for( i = 1; i <= M.tu; i ++ ) { };      循环次数 = tu (非0元素个数) ;  
for(col = 2; col <= M.nu; col++) { }; 循环次数 = nu ;  
for( p = 1; p <= M.tu ; p ++ ) { };    循环次数 = tu ;
```

该算法的时间复杂度 = $nu + tu + nu + tu = O(nu + tu)$

快速转置算法的效率分析：

讨论：最恶劣情况是矩阵中全为非零元素，此时 $tu = nu \times mu$
而此时的时间复杂度也只是 $O(mu \times nu)$ ，并未超过传统转置算法的时间复杂度。

小结：传统转置： $O(mu \times nu)$

压缩转置： $O(nu \times tu)$

压缩快速转置： $O(nu + tu)$

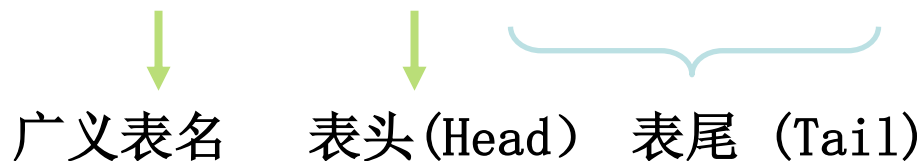
增设辅助向量，牺牲空间效率换取时间效率。

- 5.1 数组的定义
- 5.2 数组的顺序存储表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的定义
- 5.5 广义表的存储结构

1、定义：

广义表是线性表的推广，也称为列表（lists）

记为： $LS = (a_1, a_2, \dots, a_n)$


广义表名 表头(Head) 表尾 (Tail)

n 是表长

在广义表中约定：

- ① 用小写字母表示原子类型，用**大写字母**表示列表。
- ② 第一个**元素是表头**，而其余元素组成的**表**称为**表尾**；

特别提示：任何一个非空表，表头可能是原子，也可能是列表；但**表尾一定是列表**！

2、特点：

- 有次序性 一个直接前驱和一个直接后继
- 有长度 =表中元素个数
- 有深度 =表中括号的重数
- 可递归 自己可以作为自己的子表
- 可共享 可以为其他广义表所共享

讨论：广义表与线性表的区别和联系？

广义表中元素既可以是原子类型，也可以是列表；
当每个元素都为原子且类型相同时，就是线性表。

例1：求下列广义表的长度

1) $A = ()$

$n=0$ ，因为A是空表

2) $B = (e)$

$n=1$ ，表中元素e是原子

3) $C = (a, (b, c, d))$

$n=2$ ，a为原子，(b,c,d)为子表

4) $D = (A, B, C)$

$n=3$ ，3个元素都是子表

5) $E = (a, E)$

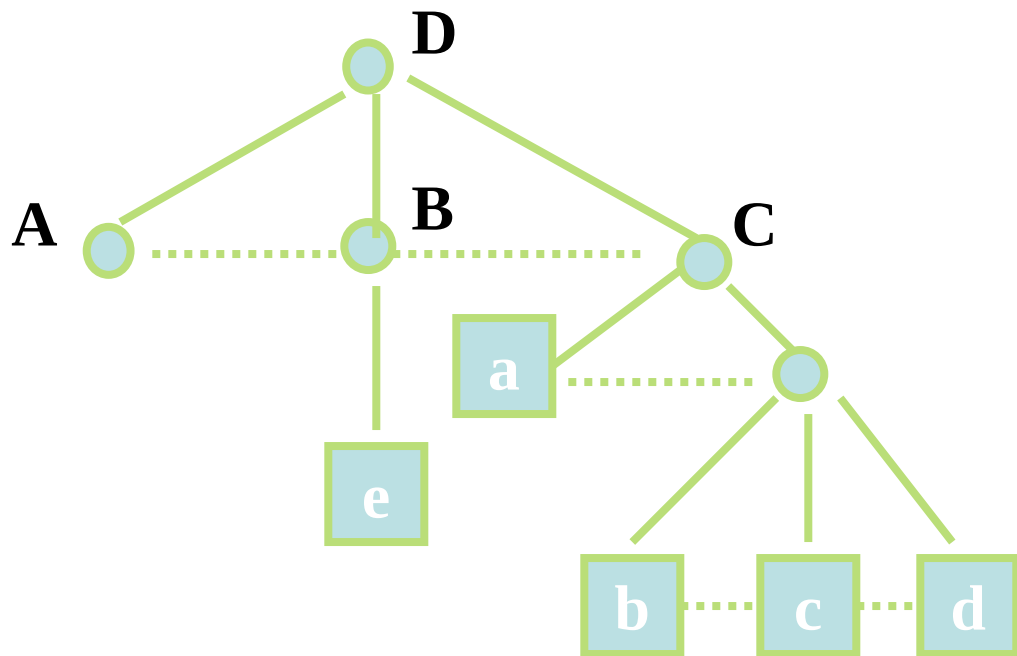
$n=2$ ，a为原子，E为子表

$D = (A, B, C) = ((), (e), (a, (b, c, d)))$ ，共享表

$E = (a, E) = (a, (a, E)) = (a, (a, (a, \dots)))$ ，E为递归表

例2：试用图形表示下列广义表（设●代表子表，■代表元素）

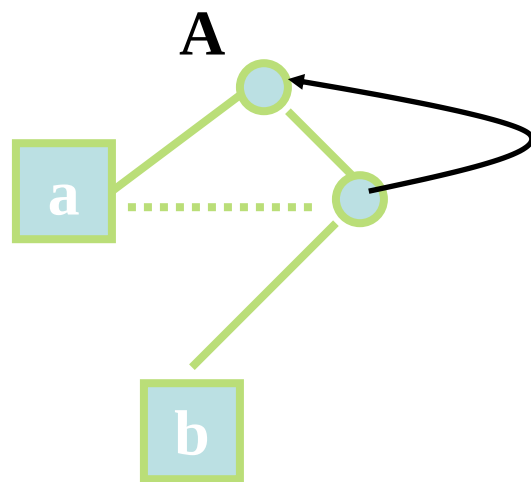
① $D=(A,B,C)=((),(e),(a, (b,c,d)))$



①的长度为3，深度为3

深度=括号的重数=●结点的层数

② $A=(a, (b, A))$



②的长度为2，深度为 ∞

广义表的抽象数据类型定义

ADT Glist{

数据对象： $D = \{ e_i \mid i=1,2,\dots,n;n \geq 0; e_i \in \text{AtomSet 或 Glist} \}$

数据关系： $R_1 = \{ \langle e_{i-1}, e_i \rangle \mid e_{i-1}, e_i \in D, 2 \leq i \leq n \}$

基本操作：

InitGList(&L);

操作结果：创建空的广义表L.

CreateGList(&L,S);

初始条件：S是广义表的书写形式串。

操作结果：由S创建广义表L.

.....

}ADT Glist

两种特殊的基本操作

GetHead (L) ——取表头(可能是原子或列表)

初始条件：广义表L存在。

操作结果：取广义表L的头。

GetTail (L) ——取表尾(一定是列表)

初始条件：广义表L存在。

操作结果：取广义表L的尾。

例：求下列广义表操作的结果

1. GetTail 【(b, k, p, h)】 = (k, p, h) ;

2. GetHead 【((a,b), (c,d))】 = (a,b) ;

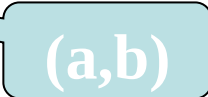
3. GetTail 【((a,b), (c,d))】 = ((c,d)) ;

4. GetTail 【 GetHead 【((a,b),(c,d))】 】 = (b) ;

5. GetTail 【 (e) 】 = () ;

6. GetHead 【 (()) 】 = () .

7. GetTail 【 (()) 】 = () .



(a,b)

- 5.1 数组的定义
- 5.2 数组的顺序存储表示和实现
- 5.3 矩阵的压缩存储
- 5.4 广义表的定义
- 5.5 广义表的存储结构

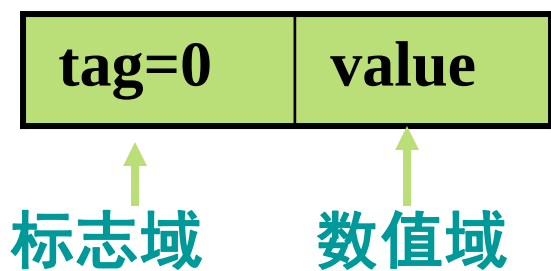
由于广义表的元素可以是不同结构（原子或列表），难以用顺序存储结构表示，**通常用链式结构**，每个**元素**用一个结点表示。

注意：列表的“元素”还可以是列表，所以结点可能有两种形式

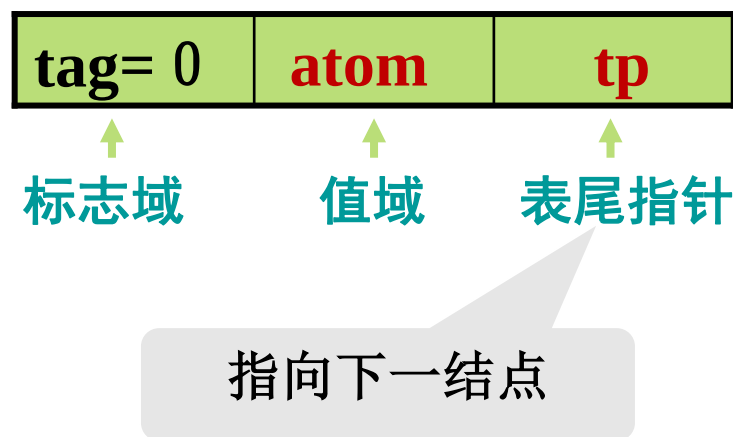
1. 原子结点

表示原子，可设2个域或3个域，依习惯而选。

法1：标志域，数值域

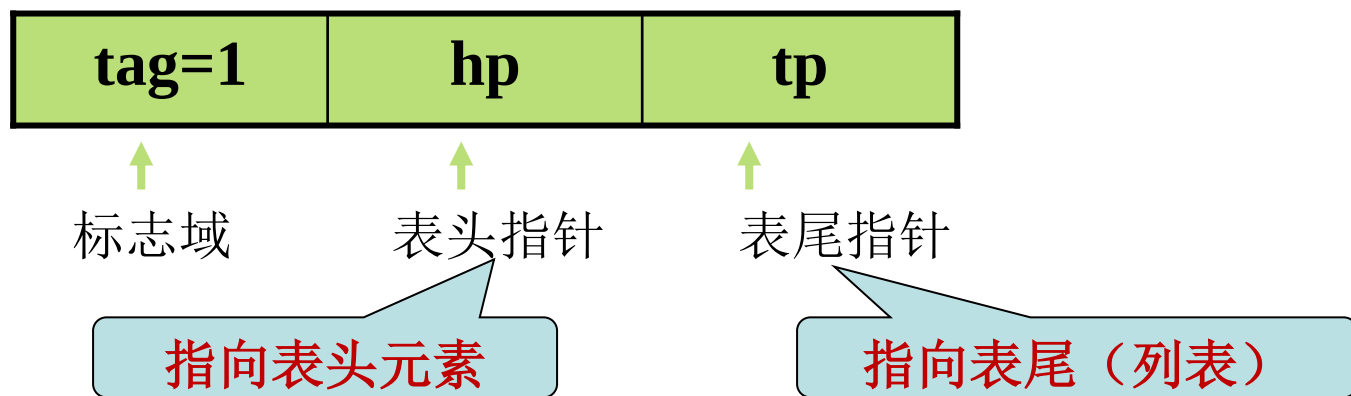


法2：标志域、值域、表尾指针



2. 表结点:

表示列表，若表不空，则可分解为表头和表尾，用3个域表示：标志域，表头指针，表尾指针。



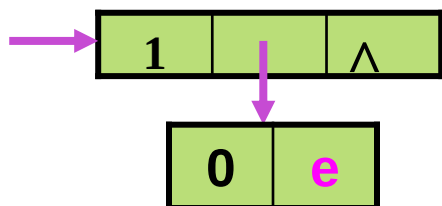
5.5

广义表的存储结构

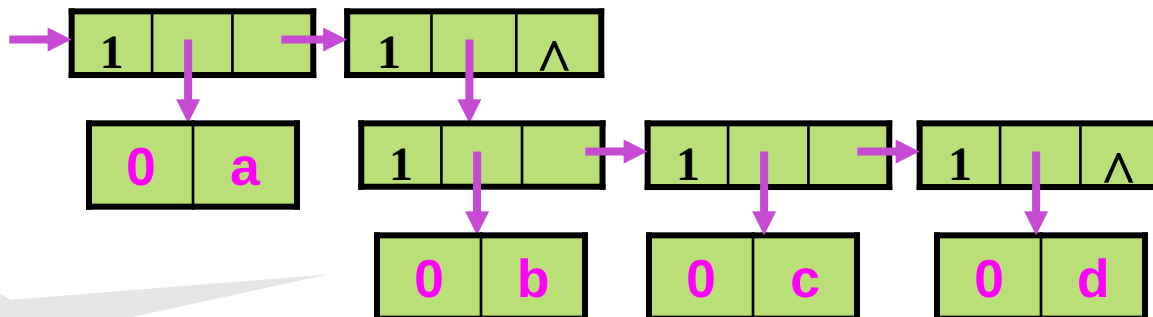
例： ① $A = ()$ $A = \text{NULL}$ $\rightarrow$

1	$\wedge$	$\wedge$
---	----------	----------

② $B = (e)$

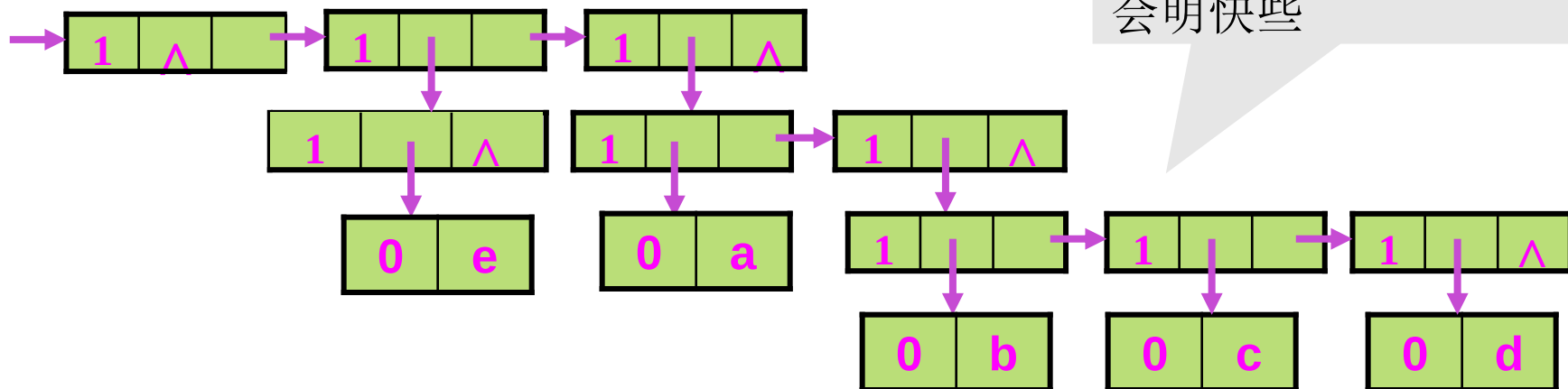


③ $C = (a, (b, c, d))$

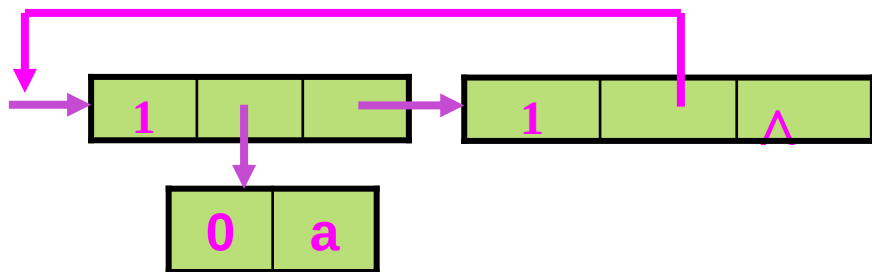


原子结点的第
一种表示法

④ $D = (A, B, C) = ((), (e), (a, (b, c, d)))$



⑤ $E = (a, E)$



1. 数组可视为一种广义线性表；
2. 数组的存储有行地址优先和列地址优先两种不同的顺序；
3. 对于稀疏矩阵，有较好的压缩存储和运算方法；
4. 广义表（列表）是线性表的推广，也是一种线性结构；
5. 任何一个非空表，表头可能是原子，也可能是列表；但表尾一定是列表。

线性结构内容结束

- 5.8
- 5.11
- 5.21
- 5.26